

- [Elma SMS Platform — Architecture & Technical Documentation](#)
 - [Table of Contents](#)
 - [1. System Overview](#)
 - [Design Principles](#)
 - [2. Component Inventory](#)
 - [3. Channel Entry Points](#)
 - [3.1 App Channel \(Mobile\)](#)
 - [3.2 USSD Channel](#)
 - [3.3 Web Channel](#)
 - [4. Message Flow — App & USSD Channels](#)
 - [Step-by-step](#)
 - [5. Message Flow — Web Channel](#)
 - [6. Queue Naming Convention & Routing Strategy](#)
 - [Queue Name Format](#)
 - [Rationale](#)
 - [Queue Durability](#)
 - [7. RabbitMQ Processor Workers](#)
 - [Processor Responsibilities](#)
 - [Scaling](#)
 - [8. SMS Splitter Gateway](#)
 - [MNO Resolution](#)
 - [Responsibilities](#)
 - [9. MNO Passthrough API & SMSC](#)
 - [Passthrough API](#)
 - [MNO SMSC Module](#)
 - [Protocol Support](#)
 - [10. Deployment Topology](#)
 - [11. Non-Functional Characteristics](#)
 - [Throughput](#)
 - [Durability](#)
 - [Observability](#)
 - [12. Failure Modes & Resilience](#)
 - [13. Glossary](#)

Elma SMS Platform — Architecture & Technical Documentation

Version: 1.0

Scope: End-to-end SMS delivery architecture

Last Updated: June 2026

Table of Contents

1. [System Overview](#)
 2. [Component Inventory](#)
 3. [Channel Entry Points](#)
 4. [Message Flow — App & USSD Channels](#)
 5. [Message Flow — Web Channel](#)
 6. [Queue Naming Convention & Routing Strategy](#)
 7. [RabbitMQ Processor Workers](#)
 8. [SMS Splitter Gateway](#)
 9. [MNO Passthrough API & SMSC](#)
 10. [Deployment Topology](#)
 11. [Non-Functional Characteristics](#)
 12. [Failure Modes & Resilience](#)
 13. [Glossary](#)
-

1. System Overview

The Elma SMS Platform is a multi-channel, multi-country SMS delivery system. It accepts inbound SMS trigger requests from three distinct channel types, routes them through an asynchronous message queue, and delivers them to Mobile Network Operators (MNOs) via SMPP or SDP protocols.

Design Principles

- **Async-first delivery:** all SMS delivery is decoupled from the initiating HTTP request via RabbitMQ, preventing back-pressure from slow MNOs from degrading the transaction layer.
- **Queue-per-segment isolation:** each combination of country, bank, and message category gets its own named queue and dedicated consumer, enabling per-

segment throttling, prioritisation, and independent scaling.

- **Protocol abstraction:** the MNO Passthrough API abstracts SMPP/SDP complexity away from the business layer; upstream components always issue a normalised HTTP call.
- **Channel parity:** regardless of whether an SMS originates from App, USSD, or Web, it converges on the same Queue API → RabbitMQ → Processor pipeline, ensuring consistent delivery behaviour.

2. Component Inventory

#	Component	Runtime	Host	Role
1	App Channel	Native mobile	Device / cloud	Initiates transactions from mobile clients
2	USSD Channel	Telecom gateway	Carrier / hosted	Initiates transactions via USSD sessions
3	Web Channel	Browser	Cloud / CDN	Initiates SMS directly via SMS API
4	Elma Core	C# (.NET)	IIS (Windows Server)	Central transaction engine; orchestrates downstream calls
5	SMS API	C# (.NET)	Docker container	Dedicated HTTP entry point for the Web channel
6	RabbitMQ API Wrapper	C# (.NET)	Docker container	Normalises publish calls; resolves queue names; writes to RabbitMQ
7	RabbitMQ	RabbitMQ broker	Docker / VM	Message broker; durably persists SMS payloads in named queues
8	Queue Processor (×N)	C# Worker Service	Docker container (one per queue)	Reads from a single named queue; forwards to SMS Splitter Gateway

#	Component	Runtime	Host	Role
9	SMS Splitter Gateway	C# (.NET)	IIS (Windows Server)	Inspects destination MSISDN; routes to the correct MNO Passthrough
10	MNO Passthrough API	Node.js	PM2 (Linux)	Per-MNO HTTP-to-SMPP/SDP adapter
11	MNO SMSC	Node.js	Same PM2 process as Passthrough	SMPP client / SDP client; delivers to MNO network

3. Channel Entry Points

The platform supports three inbound channels. Each has a different initial call path but converges on the same asynchronous queue pipeline.

3.1 App Channel (Mobile)

The mobile application calls **Elma Core** directly via a REST API. Elma Core validates the transaction, determines whether an SMS notification is required, and — if so — calls the Queue API Wrapper to enqueue the message. The HTTP response to the app is returned as soon as the transaction is processed; SMS delivery is asynchronous.

Entry call: `POST /api/transactions` → Elma Core

3.2 USSD Channel

The USSD gateway also calls **Elma Core** directly. The flow mirrors the App channel exactly. USSD sessions are typically short-lived, so Elma Core returns the USSD response immediately; any resulting SMS is enqueued and delivered asynchronously.

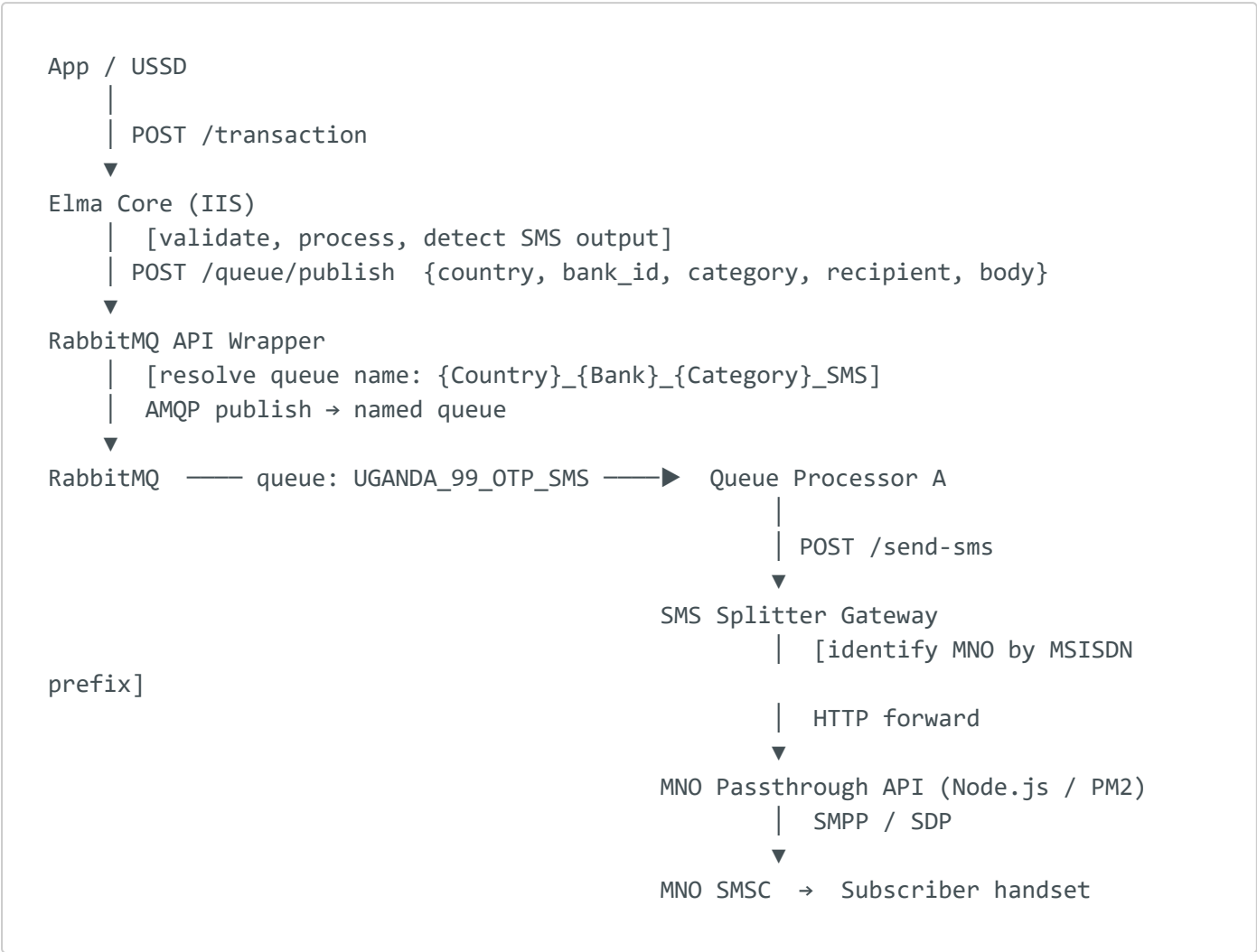
Entry call: `POST /api/ussd/session` → Elma Core

3.3 Web Channel

The Web channel bypasses Elma Core and calls the **SMS API** directly. This is the appropriate path when the web application has already completed its own transaction logic and simply needs to trigger an SMS. The SMS API normalises the request and forwards it to the Queue API Wrapper.

Entry call: `POST /api/sms/send` → SMS API → Queue API Wrapper

4. Message Flow — App & USSD Channels



Step-by-step

Step 1 — Transaction trigger

App or USSD posts a transaction request to Elma Core. Elma Core authenticates the

request, enforces business rules, and processes the transaction.

Step 2 — SMS output detection

Elma Core evaluates whether the transaction outcome requires an SMS notification (e.g. OTP dispatch, balance alert, confirmation). If yes, it constructs an SMS payload containing: recipient MSISDN, message body, country code, bank identifier, and message category.

Step 3 — Enqueue via Queue API Wrapper

Elma Core calls the Queue API Wrapper with the SMS payload. The wrapper resolves the target queue name (see Section 6), opens an AMQP channel to RabbitMQ, publishes the message to the named queue with `delivery_mode=2` (persistent), and returns a 200 ACK to Elma Core.

Step 4 — Elma Core responds to channel

Elma Core returns a `202 Accepted` to the App or USSD gateway. At this point, the transaction is complete from the channel's perspective; SMS delivery is fully async.

Step 5 — Queue Processor consumes

The Queue Processor bound to the relevant queue picks up the message. Each processor is a long-running C# Worker Service that maintains a single AMQP consumer subscription. The processor calls the SMS Splitter Gateway with the normalised SMS payload.

Step 6 — Splitter routes to MNO

The SMS Splitter Gateway inspects the destination MSISDN, resolves the MNO using a prefix lookup table, and forwards the request to the appropriate MNO Passthrough API instance.

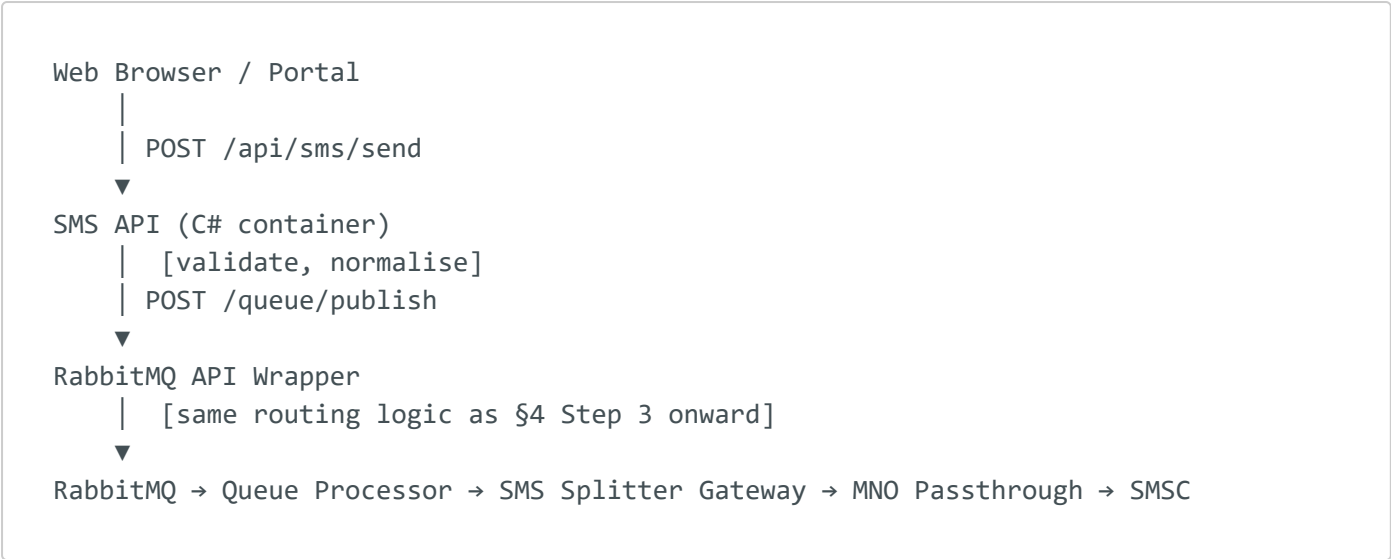
Step 7 — Passthrough delivers to SMSC

The MNO Passthrough API (Node.js, managed by PM2) translates the HTTP request into an SMPP `submit_sm` PDU (or an SDP HTTP POST, depending on the MNO's interface). The MNO SMSC module (co-located on the same Node.js thread) manages the SMPP session, handles sequence numbers, and processes `submit_sm_resp` and `deliver_sm` (delivery receipts).

Step 8 — Acknowledgement & receipt

On receipt of a delivery report from the MNO, the passthrough returns the status upstream. The Queue Processor ACKs the message in RabbitMQ, removing it from the queue.

5. Message Flow — Web Channel



The Web channel diverges from App/USSD at the entry point only. Once the payload reaches the Queue API Wrapper, the flow is identical. The SMS API container exists to:

- Provide a dedicated, independently scalable endpoint for web-originated SMS.
- Apply web-specific validation (e.g. API key authentication, rate limiting per web client).
- Allow the web integration to be modified or versioned without touching Elma Core.

6. Queue Naming Convention & Routing Strategy

Queue Name Format

```
{COUNTRY}_{BANK_ID}_{CATEGORY}_SMS
```

Examples:

Country	Bank ID	Category	Queue Name
UG (Uganda)	99	OTP	UGANDA_99_OTP_SMS
KE (Kenya)	12	PROMO	KENYA_12_PROMO_SMS

Country	Bank ID	Category	Queue Name
TZ (Tanzania)	07	ALERT	TANZANIA_07_ALERT_SMS

Rationale

Segmenting queues by country, bank, and category delivers several operational advantages:

Independent scaling: A spike in OTP messages for Bank 99 in Uganda does not affect PROMO message throughput for Bank 12 in Kenya. Each queue has its own processor with its own concurrency settings.

Priority tuning: OTP queues can be configured with higher consumer prefetch counts and more aggressive retry policies than PROMO queues, reflecting the business criticality of each category.

Fault isolation: If the MNO for a given country is degraded, only the queues for that country back up. Other countries continue processing normally.

Auditability: Queue depth per segment gives operations teams an instant view of which bank/country/category is experiencing delivery delays.

Queue Durability

All queues are declared as `durable: true` and messages are published with `delivery_mode: 2` (persistent). This ensures messages survive RabbitMQ broker restarts.

7. RabbitMQ Processor Workers

Each Queue Processor is a **.NET Worker Service** packaged as a Docker container. One container instance is deployed per queue.

Processor Responsibilities

- Maintain a single long-lived AMQP consumer subscription to its assigned queue.
- Deserialise the SMS payload from the queue message body.
- Call the SMS Splitter Gateway via HTTP POST.
- On HTTP 2xx response: ACK the RabbitMQ message (removes it from queue).
- On HTTP 4xx response: NACK with `requeue: false` (routes to dead-letter queue for manual investigation).
- On HTTP 5xx / timeout: NACK with `requeue: true` up to a configured retry limit; thereafter route to dead-letter queue.

Scaling

Because each processor is a separate container, scaling is a matter of adjusting the number of container replicas for a given queue. In high-volume scenarios, multiple replicas can share a single queue using a round-robin consumer model.

8. SMS Splitter Gateway

The SMS Splitter Gateway is a **C# API hosted on IIS**. It acts as the single internal endpoint for all SMS delivery requests from the processor layer.

MNO Resolution

The gateway maintains an MSISDN prefix routing table (e.g. `+25677` → Airtel Uganda, `+25676` → MTN Uganda, `+25470` → Safaricom Kenya). On each inbound request, it:

1. Extracts the destination MSISDN.
2. Looks up the longest matching prefix in the routing table.
3. Forwards the request to the resolved MNO Passthrough API endpoint.

Responsibilities

- Decouples the processor layer from MNO-specific endpoint URLs.
- Centralises routing table management (prefix updates do not require processor redeployment).
- Provides a single point for logging and metrics on per-MNO throughput.

- Handles MNO-level fallback (if the primary MNO endpoint is unreachable, optionally reroute to a secondary).
-

9. MNO Passthrough API & SMSC

Passthrough API

Each MNO Passthrough is a **Node.js HTTP server** managed by PM2. One passthrough process exists per MNO integration.

Responsibilities:

- Accept the normalised HTTP SMS request from the Splitter Gateway.
- Translate the payload into the MNO-specific protocol message (SMPP PDU or SDP HTTP body).
- Submit to the co-located SMSC module.
- Return the delivery status to the Splitter Gateway.

MNO SMSC Module

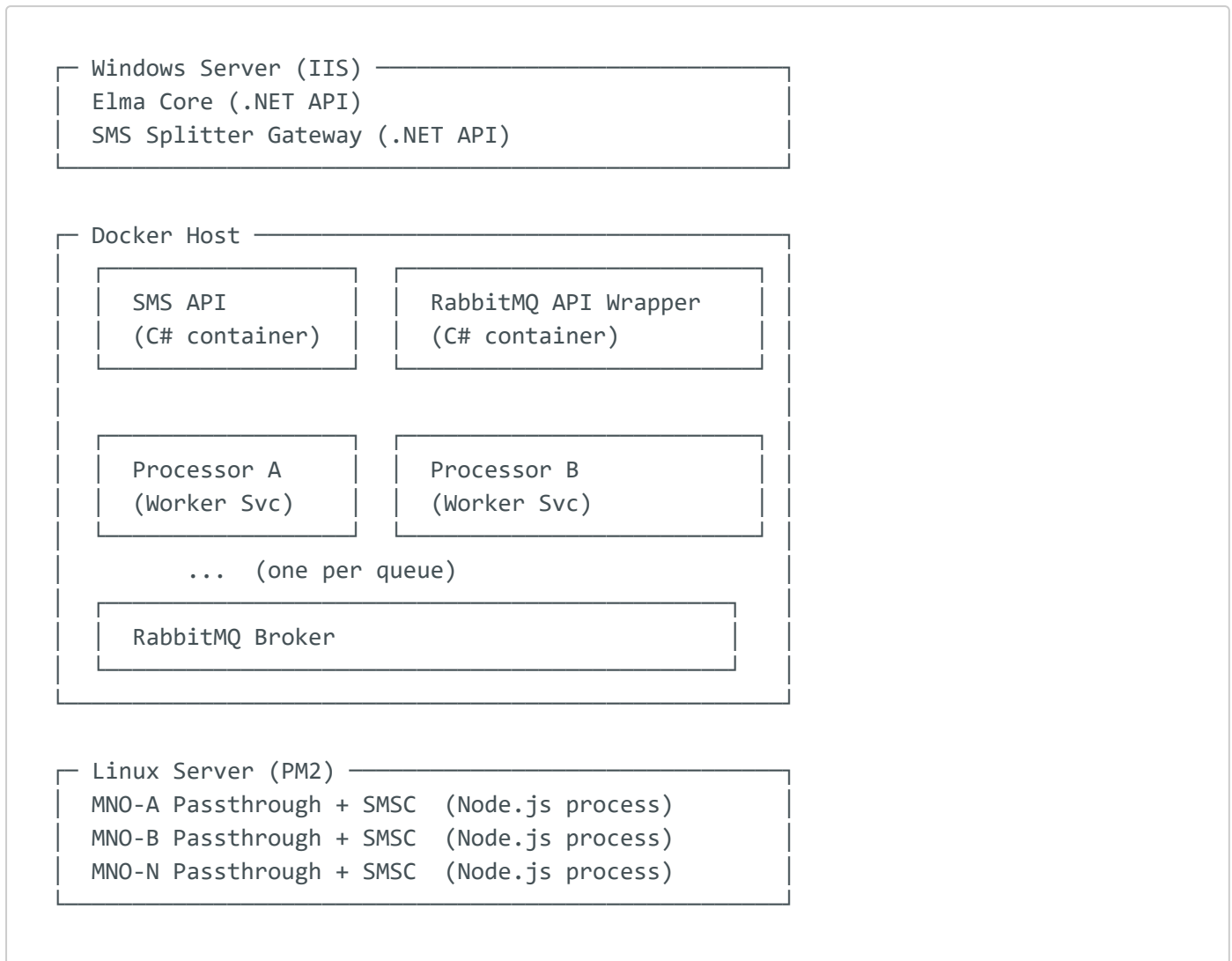
The SMSC module runs **on the same Node.js thread** as the Passthrough API (not a separate process). It manages:

- **SMPP connections:** maintains a persistent TCP session to the MNO SMSC, handles bind/rebind, enquire_link keepalives, and window size management.
- **SDP connections:** for MNOs that expose an HTTP/HTTPS SDP interface instead of SMPP, the module handles authentication tokens and request signing.
- **Sequence tracking:** assigns and tracks sequence numbers for `submit_sm` PDUs, correlates `submit_sm_resp` ACKs, and processes inbound `deliver_sm` delivery receipts.
- **Reconnection logic:** automatically rebinds on session loss with exponential backoff.

Protocol Support

Protocol	Use Case
SMPP 3.4	Most African MNO integrations
SDP (HTTP)	MNOs that expose a REST/SOAP SMS gateway

10. Deployment Topology



11. Non-Functional Characteristics

Throughput

- **Queue Processor:** throughput per processor is bounded by the SMS Splitter Gateway response time and the consumer prefetch setting. Each processor can be scaled horizontally via additional container replicas sharing the same queue.

- **SMS Splitter Gateway:** stateless HTTP handler; scales vertically (IIS thread pool) or can be put behind a load balancer.
- **MNO Passthrough:** throughput is bounded by the SMPP window size negotiated with the MNO (typically 10–100 outstanding PDUs). PM2 can run multiple instances for high-volume MNOs.

Durability

- All RabbitMQ queues and messages are persistent (**durable**, **delivery_mode: 2**).
- Dead-letter queues capture failed messages for reprocessing without data loss.

Observability

- **Queue depth:** RabbitMQ management UI / API exposes per-queue depth; alerting can be set on thresholds.
- **Delivery latency:** timestamped at enqueue and at SMSC ACK; stored in the delivery log for SLA reporting.
- **SMPP session health:** the SMSC module's `enquire_link` cycle doubles as a liveness probe.

12. Failure Modes & Resilience

Failure	Impact	Mitigation
Elma Core unavailable	App/USSD transactions fail; no SMS generated	IIS app pool auto-restart; health-check monitoring
Queue API Wrapper down	Elma Core returns error; transaction may retry	Container restart policy; Elma Core retry with backoff
RabbitMQ broker down	No new messages enqueued; existing messages safe on disk	RabbitMQ clustering / persistent storage; restart recovers queues
Queue Processor crash	Messages remain in queue (unACKed)	Worker Service restart policy; unACKed messages auto-redelivered

Failure	Impact	Mitigation
	messages requeued)	after <code>consumer_timeout</code>
SMS Splitter Gateway down	Processors receive 5xx; message NACKed and requeued	Retry with backoff; dead-letter queue after max retries
MNO Passthrough down	Splitter receives error; propagates to processor	PM2 auto-restart; splitter can route to fallback MNO if configured
SMPP session lost	MNO Passthrough pauses delivery; rebinds automatically	Exponential backoff rebind; messages queue up in RabbitMQ during outage
MNO network outage	Delivery fails at SMSC level	Dead-letter queue for manual retry; optional scheduled reprocessing job

13. Glossary

Term	Definition
AMQP	Advanced Message Queuing Protocol — the wire protocol used by RabbitMQ
SMPP	Short Message Peer-to-Peer — industry standard binary protocol for submitting SMS to an SMSC
SMSC	Short Message Service Centre — the MNO system responsible for storing and forwarding SMS messages
SDP	Service Delivery Platform — an HTTP/HTTPS-based SMS submission interface offered by some MNOs
MNO	Mobile Network Operator — the telecoms carrier (e.g. MTN, Airtel, Safaricom)
MSISDN	Mobile Station International Subscriber Directory Number — the full international phone number
OTP	One-Time Password — a message category for authentication codes

Term	Definition
PM2	Process Manager 2 — a production process manager for Node.js applications
IIS	Internet Information Services — Microsoft's web server, used to host the C# APIs
Dead-letter queue	A RabbitMQ queue that receives messages which failed all delivery attempts, for manual inspection and retry
Worker Service	A .NET background service type designed for long-running processes without an HTTP listener
Prefetch count	The number of unACKed messages a RabbitMQ consumer can hold at once — controls throughput and back-pressure